

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Expliquer le rôle d'un système d'exploitation et la distinction noyau/espace utilisateur
- Définir un processus et le distinguer d'un thread
- Décrire les 3 états d'un processus (ready, running, waiting)
- Expliquer le principe de l'ordonnancement CPU (FIFO, Round Robin)
- Naviguer dans une arborescence de fichiers et comprendre les permissions Unix
- Adapter ces connaissances pour des activités SNT et des situations d'enseignement

Plan du cours

1. Introduction : qu'est-ce qu'un système d'exploitation ? (5 min)
2. Architecture de l'OS : noyau, espace noyau, espace utilisateur (15 min)
3. Processus et threads : définitions, états, cycles de vie (20 min)
4. Ordonnancement CPU : FIFO, Round Robin, priorités (20 min)
5. Système de fichiers : arborescence, répertoires, permissions (20 min)
6. Applications pédagogiques et ligne de commande (10 min)

1. Rôle du système d'exploitation

```
graph TB
    USER["Utilisateur<br/>(applications)"] --> UI["Interface Utilisateur<br/>(GUI / CLI)"]
    UI --> OS["Système d'exploitation"]
    OS --> KERNEL["Noyau (Kernel)"]

    subgraph "Services du noyau"
        PROC["Gestion des processus<br/>Ordonnancement"]
        MEM["Gestion mémoire<br/>Mémoire virtuelle"]
        FS["Système de fichiers<br/>Stockage & permissions"]
        DEV["Gestion périphériques<br/>Pilotes (drivers)"]
        NET["Réseau<br/>TCP/IP stack"]
    end

    KERNEL --> PROC
    KERNEL --> MEM
    KERNEL --> FS
    KERNEL --> DEV
    KERNEL --> NET

    PROC --> HW["Matériel<br/>(CPU, RAM, disques, ...)"]
    MEM --> HW
```

```

FS --> HW
DEV --> HW
NET --> HW

style OS fill:#e74c3c,color:#fff
style KERNEL fill:#3498db,color:#fff
style HW fill:#95a5a6,color:#fff

```

Définition : Un système d'exploitation (OS) est un logiciel qui fait l'interface entre le matériel et les applications. Il gère les ressources de l'ordinateur et fournit des services aux programmes.

Rôles principaux :

1. Abstraction matérielle : cacher la complexité du matériel aux applications
2. Multiplexage des ressources : partager le CPU, la mémoire, les disques entre plusieurs programmes
3. Isolation et protection : empêcher un programme d'accéder aux données d'un autre
4. Gestion des entrées/sorties : communiquer avec les périphériques via des pilotes (drivers)

Exemples d'OS : Windows 10/11 (Microsoft), Linux (Ubuntu, Debian, Fedora), macOS (Apple), Android (Google), iOS (Apple).

Noyau (Kernel) : cœur de l'OS, chargé de la gestion des ressources. Il s'exécute en mode noyau (accès total au matériel). Les applications s'exécutent en mode utilisateur (accès restreint). Les appels système (syscalls) permettent de passer du mode utilisateur au mode noyau.

2. Processus et threads

2.1 Définition d'un processus

Processus : un programme en cours d'exécution, avec son propre espace mémoire (code, données, pile, tas), ses fichiers ouverts, et son contexte d'exécution.

Chaque processus possède :

- Un PID (Process IDentifier) — numéro unique
- Un état (ready, running, waiting)
- Un compteur ordinal (PC) — prochaine instruction à exécuter
- Des registres (contexte CPU)
- Une table des fichiers ouverts
- Un espace mémoire privé

2.2 Threads

Thread (fil d'exécution) : unité d'exécution plus légère qu'un processus. Un processus peut contenir plusieurs threads qui partagent le même espace mémoire.

Différence processus vs thread :

Caractéristique	Processus	Thread
-----------------	-----------	--------

Espace mémoire	Indépendant (isolé)	Partagé avec les threads du même processus
Création	Plus lourde (copie des ressources)	Plus légère
Communication	IPC (pipes, sockets, signaux)	Mémoire partagée directe
Isolation	Forte (un crash n'affecte pas les autres)	Faible (un crash peut tout détruire)
Commutation	Plus coûteuse	Plus rapide

2.3 Les 3 états d'un processus

```
stateDiagram-v2
[*] --> NOUVEAU : Création (fork)
NOUVEAU --> READY : Prêt (chargé en mémoire)
READY --> RUNNING : Élu par l'ordonnanceur
RUNNING --> READY : Interruption (temps écoulé)
RUNNING --> WAITING : Attente d'une ressource (E/S)
WAITING --> READY : Ressource disponible
RUNNING --> TERMINE : Fin du processus (exit)
TERMINE --> [*]
```

```
note right of READY : En attente de CPU
note right of RUNNING : En cours d'exécution
note right of WAITING : Bloqué (I/O, sleep)
```

Transitions d'états :

- Nouveau → Prêt (Ready) : le processus est créé et placé dans la file des prêts
- Prêt → Élu (Running) : l'ordonnanceur sélectionne un processus prêt et lui alloue le CPU
- Élu → Prêt : le quantum de temps est écoulé, le processus est remis dans la file des prêts
- Élu → Bloqué (Waiting) : le processus demande une opération d'E/S (lecture disque, attente réseau)
- Bloqué → Prêt : l'opération d'E/S est terminée, le processus retourne dans la file des prêts
- Élu → Terminé : le processus a fini son exécution (appel à `exit()`)

Changement de contexte (context switch) : Lorsque le CPU passe d'un processus à un autre, il doit sauvegarder le contexte du processus sortant (registres, PC) et restaurer celui du processus entrant. Cette opération a un coût (quelques microsecondes).

3. Ordonnancement CPU

L'ordonnanceur (scheduler) décide quel processus prêt obtient le CPU. Objectifs : équité, efficacité, temps de réponse minimal.

3.1 FIFO (First In, First Out)

```
graph LR
subgraph "File des processus prêts (FIFO)"
P1["P1<br/>Arrivé à t=0"] --> P2["P2<br/>Arrivé à t=2"]
P2 --> P3["P3<br/>Arrivé à t=4"]
P3 --> P4["P4<br/>Arrivé à t=6"]
end

subgraph "Exécution sur le CPU"
EXEC["P1 (0-5) | P2 (5-8) | P3 (8-10) | P4 (10-12)"]
end
```

P1 --> EXEC

Principe : Le premier processus arrivé est le premier servi. Pas de préemption (non préemptif). Une fois qu'un processus a le CPU, il le garde jusqu'à ce qu'il se termine ou se bloque.

Avantages : Simple à implémenter, équitable pour les processus longs.

Inconvénients : Temps d'attente moyen élevé pour les processus courts (effet de convoi).

3.2 Round Robin (RR)

```

gantt
title Ordonnancement Round Robin (quantum = 2 unités)
dateFormat X
axisFormat %s

section P1 (durée 6)
Tour 1 : 0, 2
Tour 2 : 6, 2
Tour 3 : 10, 2

section P2 (durée 4)
Tour 1 : 2, 2
Tour 2 : 8, 2

section P3 (durée 2)
Tour 1 : 4, 2

section P4 (durée 2)
Tour 1 : 12, 2

```

Principe : Chaque processus reçoit un quantum de temps fixe (ex: 10 ms). Si le processus ne se termine pas dans son quantum, il est remis dans la file des prêts et le suivant est exécuté.

Avantages : Équitable, temps de réponse court pour les processus interactifs.

Inconvénient : Le choix du quantum est crucial — trop court → changement de contexte trop fréquent, trop long → dégradation du temps de réponse.

Exemple de calcul (Round Robin, quantum = 2) :

Processus	Durée	Début	Fin	Temps d'attente
P1	6	0	12	6
P2	4	2	10	4
P3	2	4	6	2
P4	2	12	14	10

Temps d'attente moyen : $(6 + 4 + 2 + 10) / 4 = 5,5$ unités de temps.

4. Système de fichiers

4.1 Arborescence des fichiers

```

graph TB
ROOT["/ (racine)"] --> BIN["/bin<br/>commandes essentielles"]
ROOT --> HOME["/home<br/>répertoires utilisateurs"]
ROOT --> ETC["/etc<br/>configuration système"]

```

```

ROOT --> USR["/usr<br/>programmes installés"]
ROOT --> VAR["/var<br/>données variables (logs)"]
ROOT --> TMP["/tmp<br/>fichiers temporaires"]
ROOT --> DEV["/dev<br/>périphériques"]

HOME --> USER1["/home/madani<br/>(votre répertoire)"]
USER1 --> DOC["Documents"]
USER1 --> TEL["Téléchargements"]
USER1 --> BUR["Bureau"]
USER1 --> MUSIC["Musique"]
USER1 --> IMG["Images"]

style ROOT fill:#e74c3c,color:#fff
style HOME fill:#3498db,color:#fff
style USER1 fill:#2ecc71,color:#fff

```

Structure : Arborescence enracinée (Unix/Linux) ou avec lettres de lecteurs (Windows).

Chemin absolu : commence par / (racine). Ex: /home/madani/Documents/cours.md

Chemin relatif : par rapport au répertoire courant. Ex: ../Documents/cours.md

Raccourcis :

- . : répertoire courant
- .. : répertoire parent
- ~ : répertoire personnel de l'utilisateur

4.2 Commandes de base (terminal Linux)

Commande	Rôle	Exemple
<code>pwd</code>	Afficher le répertoire courant	<code>pwd</code>
<code>ls</code>	Lister les fichiers	<code>ls -la</code>
<code>cd</code>	Changer de répertoire	<code>cd D</code>
<code>mkdir</code>	Créer un répertoire	<code>mkdir</code>
<code>cp</code>	Copier un fichier	<code>cp s</code>
<code>mv</code>	Déplacer/renommer	<code>mv a</code>
<code>rm</code>	Supprimer	<code>rm fi</code>
<code>cat</code>	Afficher le contenu	<code>cat f</code>
<code>chmod</code>	Modifier les permissions	<code>chm</code>

4.3 Permissions (Unix)

```

graph TD
subgraph "Permissions d'un fichier"
PERM["-rwxr-xr--"]
TYP["Type :<br/>- fichier<br/>d répertoire<br/>l lien"]
U["User (owner)<br/>rwx<br/>(lecture, écriture, exécution)"]
G["Group<br/>r-x<br/>(lecture, exécution)"]
O["Others<br/>r--<br/>(lecture seule)"]
end

TYP --> PERM
PERM --> U --> G --> O

```

Représentation symbolique : -rwxr-xr--

- Position 1 : type (- fichier, d répertoire, l lien symbolique)
- Positions 2-4 : droits du propriétaire (r=read, w=write, x=execute)
- Positions 5-7 : droits du groupe
- Positions 8-10 : droits des autres

Représentation octale :

- $r = 4, w = 2, x = 1$
- `chmod 755 → rwxr-xr-x`
- `chmod 644 → rw-r--r--`

Exemple pour l'enseignement : Protéger un fichier de notes confidentiel :

```
chmod 600 notes_eleves.txt # seul le propriétaire peut lire/écrire
```

QCM d'auto-évaluation

1. Quel est le rôle principal du noyau (kernel) d'un OS ?
a) Afficher une interface graphique b) Gérer les ressources matérielles ✓ c) Naviguer sur Internet
2. Dans quel état un processus attend-il que le CPU lui soit alloué ?
a) Running b) Waiting c) Ready ✓
3. Quel algorithme d'ordonnancement utilise un quantum de temps fixe ?
a) FIFO b) Round Robin ✓ c) SJF
4. Que signifie le droit x sur un répertoire ?
a) Pouvoir le supprimer b) Pouvoir le traverser (y accéder) ✓ c) Pouvoir le renommer
5. Quelle commande permet d'afficher le répertoire courant sous Linux ?
a) cd b) ls c) pwd ✓

Conclusion

Le système d'exploitation est un logiciel fondamental qui gère toutes les ressources de l'ordinateur. La distinction noyau/espace utilisateur garantit la stabilité et la sécurité. Les processus et threads sont les unités d'exécution de base ; leur ordonnancement (FIFO, Round Robin) détermine la réactivité du système. Le système de fichiers organise les données dans une arborescence et les permissions Unix permettent de contrôler les accès.

Prochaine séance : Algorithmique fondamentale — variables, conditions, boucles et complexité.

Note pédagogique ENS

Transposition didactique : En classe de SNT Seconde (thème « Systèmes d'exploitation ») :

- Simuler l'ordonnancement Round Robin en faisant jouer les processus par des élèves. Un élève est l'ordonnanceur et distribue un « quantum » (ex: 30 secondes) à chaque

processus.

- Activité : créer une arborescence de répertoires pour un projet de classe (avec `mkdir` en ligne de commande)
- Expliquer les permissions avec l'analogie d'une salle de classe (propriétaire = enseignant, groupe = élèves)

Lien avec les langues : Le concept d'arborescence de fichiers peut être comparé à la structure grammaticale (arbres syntaxiques). Les threads (exécution légère) peuvent être comparés aux conversations parallèles dans une classe.

Bibliographie

- Tanenbaum, A. & Bos, H. (2015). Systèmes d'exploitation : Modern Operating Systems (4e éd.). Pearson.
- Silberschatz, A., Galvin, P. & Gagne, G. (2018). Operating System Concepts (10e éd.). Wiley.
- Le livre du système Linux : <https://linuxcommand.org/>
- Dowek, G. et al. (2019). SNT Seconde. Hachette (chapitre « Systèmes d'exploitation »).
- Documentation Ubuntu : <https://doc.ubuntu-fr.org/>